

# 动态开点

```
int build(int l,int r)
{
    ++k; //k表示当前节点编号
    if(l==r)
    {
        tree[id].sum=a[l];
        return k; //把当前节点编号返回回去
    }
    int mid=(l+r)/2;
    tree[k].lson=build(l,mid);
    tree[k].rson=build(mid+1,r);
    tree[k].sum=tree[tree[k].lson].sum+tree[tree[k].rson].sum;
    return k; //把节点编号返回去
}
root=build(1,n);
```

主要用途：主席树，二维线段树，大区间修改

# 标记永久化

```
void query(int id,int l,int r,int s)//s表示路径上的标记之和
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].sum+s*(tree[id].r-tree[id].l+1);
    }
    int mid=(l+r)/2;
    if(r<=mid)
        return query(id*2,l,r,s+tree[id].lazy);
    else
        return query(id*2+1,l,r,s+tree[id].lazy);
    else
        return query(id*2+1,l,mid,s+tree[id].lazy)+query(id*2+1,mid+1,r,s+tree[id].lazy);
}
```

主要用途：二维线段树，标记不能下传

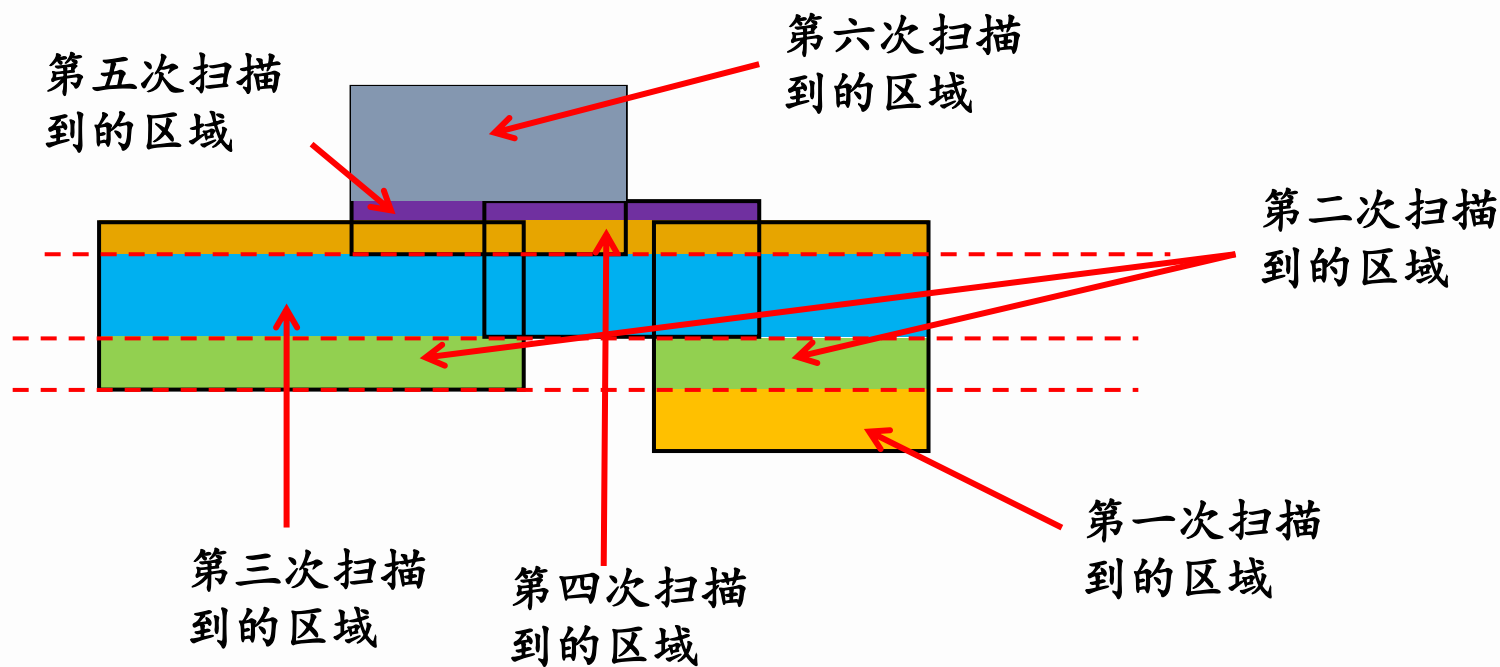
# 区间合并

```
void updatafa(int id)
{
    if(tree[id*2].lmax==tree[id*2].len)//左子树全1
        tree[id].lmax=tree[id*2].len+tree[id*2+1].lmax;
    else
        tree[id].lmax=tree[id*2].lmax;
    if(tree[id*2+1].rmax==tree[id*2+1].len)//右子树全1
        tree[id].rmax=tree[id*2+1].len+tree[id*2].rmax;
    else
        tree[id].rmax=tree[id*2+1].rmax;
    tree[id].zmax=max(tree[id*2].zmax,tree[id*2+1].zmax);
    tree[id].zmax=max(tree[id].zmax,tree[id*2].rmax+tree[id*2+1].lmax);
}
```

# 区间合并

```
void updatafa(int id)
{
    if(tree[id*2].lmax==tree[id*2].len)//左子树全1
        tree[id].lmax=tree[id*2].len+tree[id*2+1].lmax;
    else
        tree[id].lmax=tree[id*2].lmax;
    if(tree[id*2+1].rmax==tree[id*2+1].len)//右子树全1
        tree[id].rmax=tree[id*2+1].len+tree[id*2].rmax;
    else
        tree[id].rmax=tree[id*2+1].rmax;
    tree[id].zmax=max(tree[id*2].zmax,tree[id*2+1].zmax);
    tree[id].zmax=max(tree[id].zmax,tree[id*2].rmax+tree[id*2+1].lmax);
}
```

# 矩形面积之和



3、把边按照y值从小到大排序，然后从下到上扫描上下边，扫描到下边时，区间值+1，扫描到上边时，区间值-1，，用（下一条边-上一条边）\*此时区间有效长度（非0的个数）来计算面积。

# 实现过程

```
void update(int id, point a)
{
    if(tree[id].fl==a.x1&&tree[id].fr==a.x2)
    {
        tree[id].cf+=a.flag;
        js(id); // 计算覆盖长度, 重复的只算一次
        return ;
    }
    if(a.x2<=tree[id*2].fr) // 如果该线段都在左区间
        update(id*2, a);
    else if(a.x1>=tree[id*2+1].fl) // 如果都在右区间
        update(id*2+1, a);
    else // 如果两个区间各一部分
    {
        point t1=a, t2=a;
        t1.x2=tree[id*2].fr;
        update(id*2, t1);
        t2.x1=tree[id*2+1].fl;
        update(id*2+1, t2);
    }
    js(id); // 一定不要忘记更新根节点的信息
}
```

# 矩形周长之和

首先，矩形的周长分为长和宽，如果我们按照前面求面积的方法来做，宽度就不太好求，所以我们可以把矩形的长和宽分开来求解，即先求水平的，再求垂直的。

在考虑的时候，我们发现要分上底面和下底面考虑。

对于下底面的边  $[s, t]$ ，先查询  $[s, t]$  之间被覆盖的长度  $x$ ，然后  $t-s-x$  就是新增的

对于上底面的边  $[s, t]$ ，先删除  $[s, t]$  对应的边，然后再查询  $[s, t]$  之间被覆盖的长度  $x$ ，然后  $t-s-x$  就是。

# 动态区间

但是对于一些特殊的区间修改，比如区间删除，或者单点删除，或者部分单点添加，实际上线段树还是可以做的，以前的线段树，对于区间 $[l, r]$ ，那么实际存在的点的个数就是 $r-l+1$ 个，但是遇到这种点删除的情况。如果我们要让区间仍然不变，那么我们就要再维护当前区间真实节点数量，比如删除 $i$ 号节点，如果 $i$ 号节点在左边，那么左边真实儿子数量--，如果 $i$ 号节点在右边，那么右边真实儿子数量--。如果要查询区间 $[l, r]$ ，主要是去看当前节点的左右子树的真实儿子。如果 $r \leq$ 左儿子的真实儿子数量，那么就去左区间查找，如果 $l >$ 右儿子的真实儿子数量，那么就去右区间查找。



# 动态最值

```
void update(int id,int k)
{
    if(tree[id].l==tree[id].r)//既然是单点修改, 所以是修改叶子节点
    {
        tree[id].son=0;//真实儿子个数为0
        tree[id].Max=-inf;//存在负数
        tree[id].Min=inf;
        return ;
    }
    if(k<=tree[id*2].son)//和当前节点左儿子的节点数作比较
    update(id*2,k);
    else
    update(id*2+1,k-tree[id*2].son);//注意要减去左儿子数量
    tree[id].Max=max(tree[id*2].Max,tree[id*2+1].Max);
    tree[id].Min=min(tree[id*2].Min,tree[id*2+1].Min);
    tree[id].son=tree[id*2].son+tree[id*2+1].son;
}
```

# 线段树

```
node query(int id,int l,int r)
{
    //if(l==1&&r==tree[id].son)
    if(r-l+1==tree[id].son)//如果找到一个完整区间
        return tree[id];
    if(r<=tree[id*2].son)//都在左区间
        return query(id*2,l,r);
    else if(l>tree[id*2].son)//都在右区间
        return query(id*2+1,l-tree[id*2].son,r-tree[id*2].son);
    else
    {
        node t1=query(id*2,l,tree[id*2].son);
        node t2=query(id*2+1,1,r-tree[id*2].son);
        node tmp;
        tmp.Max=max(t1.Max,t2.Max);
        tmp.Min=min(t1.Min,t2.Min);
        return tmp;
    }
}
```

# 线段树

实际上线段树还有一种动态开点的办法，一般的线段树都是在最开始的时候就把线段树每个节点建好，然后再去修改查询，这里介绍一种最开始不建线段树的做法，最开始只有一个节点，如果有一次修改操作。比如单点修改，那么我就只对需要查询到的点新增节点，其他访问不到的点我不用去建节点，那么每次修改的时间辅助就是 $\log n$ 。同样访问也是一样，我要去查询这个区间，我只把需要用到的点建出来，其他点不管，最坏时间复杂为 $\log n^2$ 。和线段树点的区间没有关系。

但是这种线段树只能适用于知道最开始线段树所有节点信息的题

# 线段树

一：最开始

①

$[1, 10]$

二：修改编号为k的节点，权值为x

比如 3 5

①

$[1, 10]$

②

$[1, 5]$

③

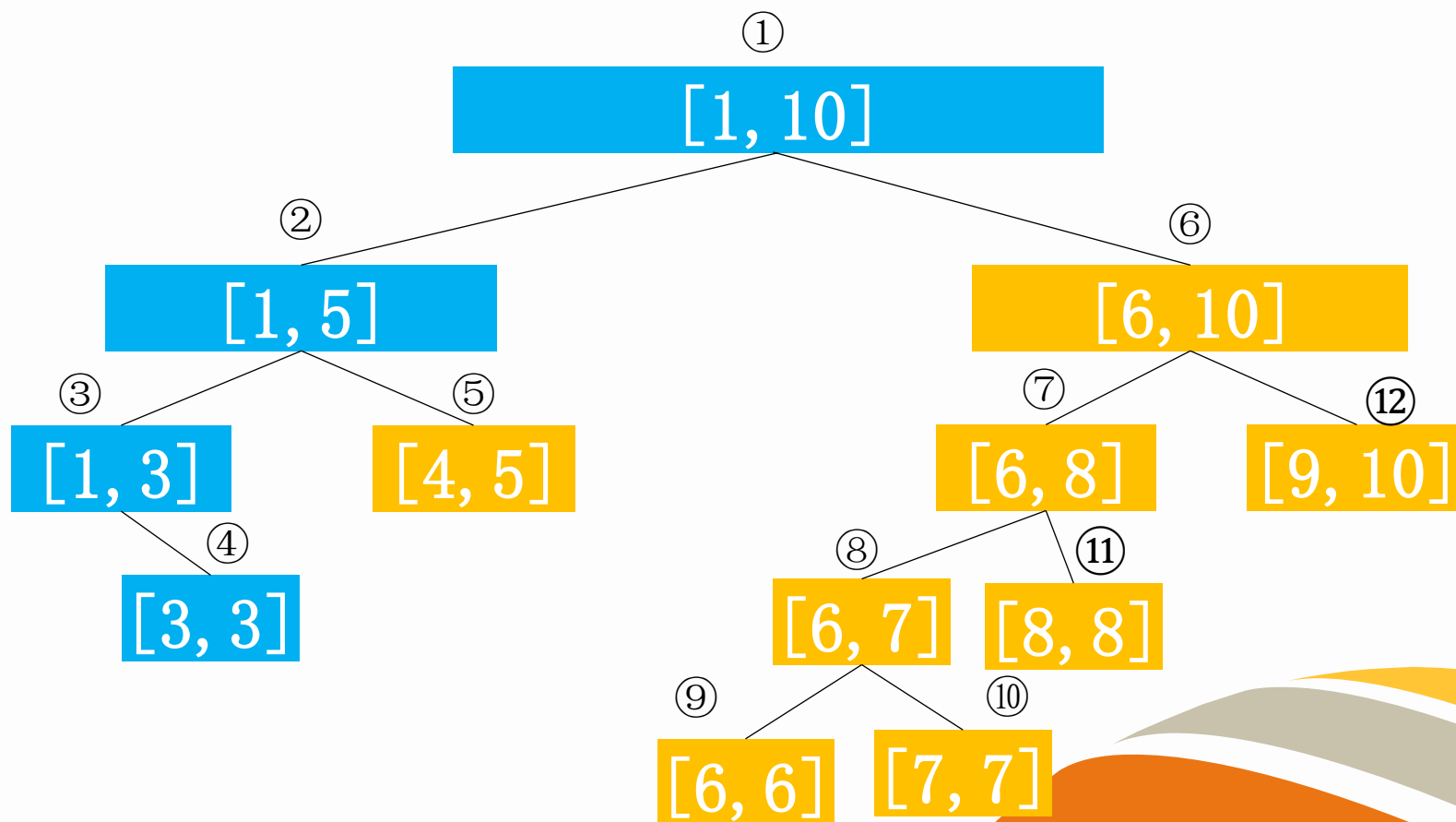
$[1, 3]$

④

$[3, 3]$

# 线段树

三：查询区间 $[2, 7]$ 的最大值



# 线段树

## 第一步：线段树建树

```
int build(int id,int l,int r)
{
    ++k;//动态开点
    tree[id].l=l;
    tree[id].r=r;
    return k;//最开始只有一个根节点
}
for(int i=1;i<=n;i++)
    root[i]=build(1,1,m);//建立n棵行线段树
```

# 线段树

## 第二步：线段树区间修改

```
void update(int id,int l,int r,int x)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        ...
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id*2])//如果左二子不存在
        tree[id].ls=build(l,mid);
    if(!tree[id*2+1])//如果右儿子不存在
        tree[id].rs=bbuild(mid+1,r);
    //标记下传
    if(r<=mid)
        update(id*2,l,r,x);
    else if(l>mid)
        update(id*2+1,l,r,x);
    else
    {
        update(id*2,l,mid,x);
        update(id*2+1,mid+1,r,x);
    }
    tree[id].Max=max(tree[tree[id].ls].Max,tree[tree[id].rs].Max);
}
update(root[i],l,r,x)//修改第i行的区间[l,r]增加一个值x
```

# 线段树

## 第三步：线段树区间查询

```
int query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].Max;
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id*2])//如果左二子不存在
        tree[id].ls=build(l,mid);
    if(!tree[id*2+1])//如果右儿子不存在
        tree[id].rs=bbuild(mid+1,r);
    //标记下传
    if(r<=mid)
        return query(id*2,l,r,x);
    else if(l>mid)
        return query(id*2+1,l,r,x);
    else
        return max(query(id*2,l,mid,x),query(id*2+1,mid+1,r,x));
}
query(root[i],l,r);
```



# 树上统计

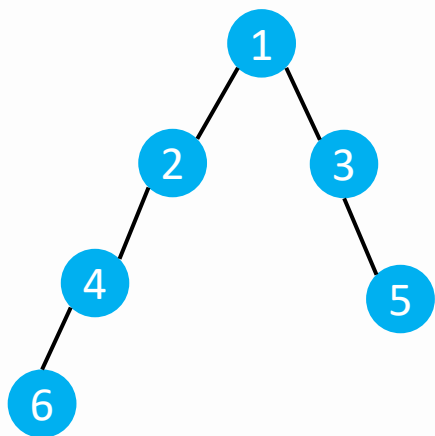
在前面我们学习了两种树上简单统计的办法：

第一种是基于不带修改的，我们可以直接用 lca 来解决。

第二种是基于两点之间的最短路径修改的，我们可以用树上差分来解决。但是树上差分也有一个问题。第一，他不能不停的询问，只能是离线的，如果在线，每次查询的时间复杂度为  $O(n)$ ，第二，他不能处理任意的修改，所以树上差分他有很大的局限性。所以我们现在要学习一种高级的数据结构来维护树上的任意修改——树链剖分。

# 树链剖分

对于树上任意两点的之间的距离，我们仅仅知道树上相邻两点间的距离，如果要求解，那么就需要把每一条边加起来，这样时间就比较慢，但是如果我们可以把一些连着的短边看做一条完整长边，那么计算总长度的时候只需要累加长边就可以了，这样就节省了时间。



比如查询6—5，如果我们先就把6—1之间这三条边看做一条完整的边，把1—5之间这两条边看做一条完整的边，那么最后计算的时候，我们只需要把这两条完整的边相加就得到了最终的答案，计算很快。

# 树链剖分

所以我们这里需要找到一种把一棵树拆分成很多条链的拆分方式，为了保证时间效率，我们必须保证任意两个点之间的链的数量最多为  $\log n$  这个级别，这样才能保证总体时间复杂度。

树链剖分的方式有很多种，常见的有轻重链剖分，虚实链剖分等等，这些剖分方式拆分成的链都能保证时间复杂度，这里先介绍一种比较简单的轻重链剖分方法。

# 树链剖分

我们先要定义几个概念：

重儿子：父亲结点的所有儿子中结点数**最多**的结点

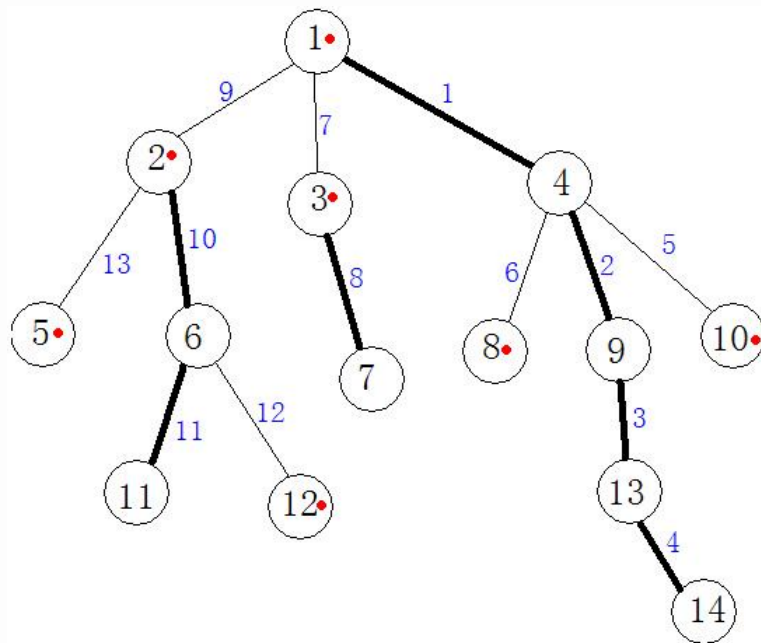
轻儿子：父亲结点除了重儿子之外的**所有**儿子结点

重边：父亲结点和重儿子相连的边

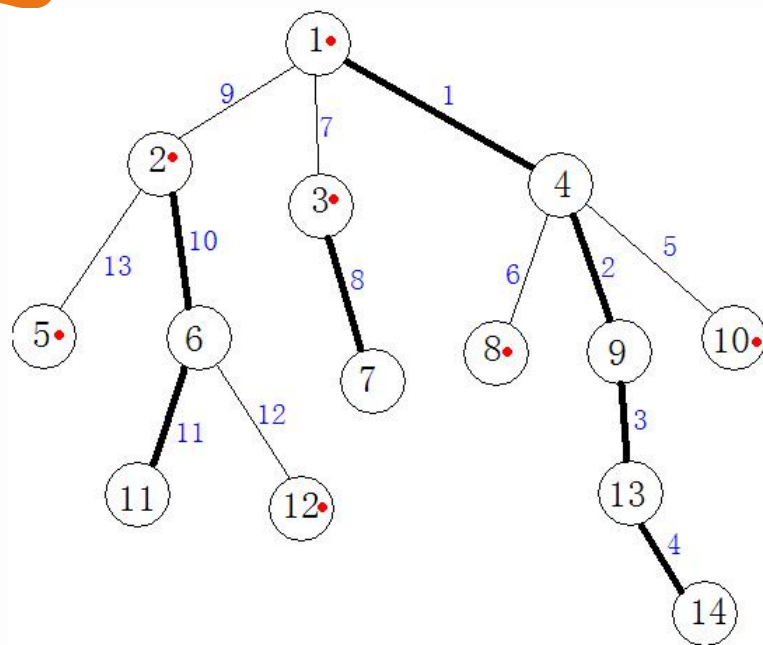
轻边：父亲结点和轻儿子相连的边

重链：由多条重边连接而成的路径

轻链：有多条轻边连接而成的路径



# 树链剖分



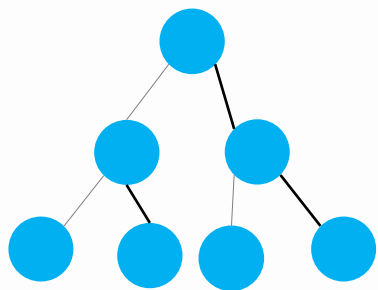
这样我们就把一棵树拆分成了多条重链和轻链。对于任意一个结点，一定存在重儿子，但是不一定存在轻儿子。

**重链可以是连续的，但是轻链一定是单独的**

比如我们查询12—14，那么我们只需要查询链12-6, 2-6, 2-1, 1-14，查询次数减少了很多次，但是我们是否可以证明如果把一个数按照上面的轻重链剖分的方法一定能保证任意两点间轻重链的数量一定是 $\log n$ 级别的呢？

# 树链剖分

对于一颗树，如果要想让他到一个结点的链数最多，那么极限情况下就是从根结点到一个深度最深的结点的边全是轻边，这样就能保证他们之间的链的条数最多，那么要构造这样一颗树，结点数至少是多少呢？



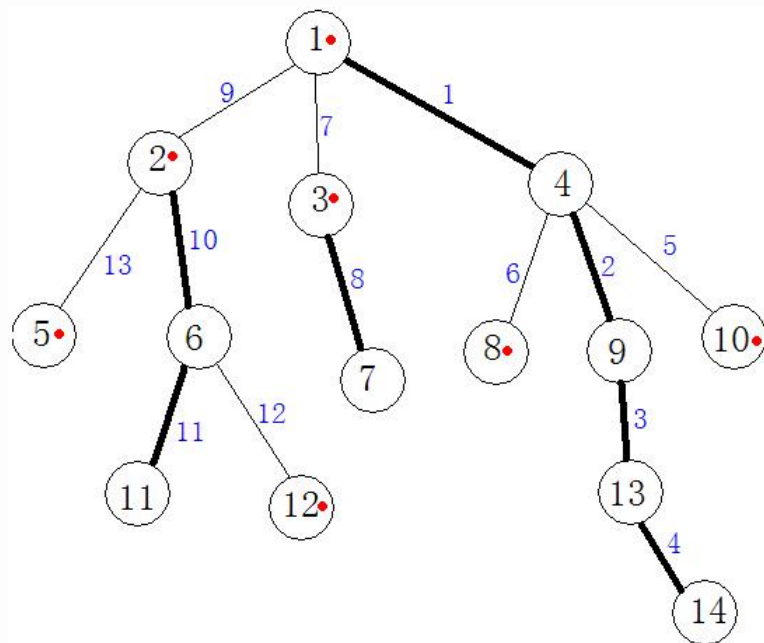
因为既然该结点是轻儿子，那么该结点的父亲结点的其他儿子数量肯定大于等于他，最坏就只有两个儿子，且另一个儿子数量和他相等。那么这样算下来，当两点之间有 $m$ 条轻边时，这棵树至少有 $2^{m+1}-1$ 个结点。

那么对于有 $n$ 个结点的数，任意两点之间的轻边数量最多 $\log n$ 级别。

# 树链剖分

那么轻重链剖分可以用来做什么呢？

轻重链剖分可以用来求解最近公共祖先，也就时lca，比如我们要求任意两个点的最近公共祖先，那么只需要这两个点中当前深度最深的点一直向上跳，如果在轻链上就向上跳一格，如果是重链就跳到该重链的顶点最终一定可以跳到他们的公共祖先上去



因为任意两点之间的链的数量最多 $\log n$ 条，所以最多向上跳 $\log n$ 次就会跳到同一个位置，所以时间复杂度也是可以接受的。

# 树链剖分

我们设每个点的深度为 $\text{deep}[x]$ ，设每个结点的父亲结点是 $\text{fa}[x]$ ，每个点所在链的顶端结点为 $\text{top}[x]$ ，重链上的点的顶点是这条重链的顶端，轻链上的点的顶点就是它本身。

但是一定要注意写法，因为重链上的点我们是当做一个整体考虑，所以比较的时候不仅仅是比较 $u, v$ 两个点的深度，而是比较这两个点所在链顶端的深度，否则就会导致错误。

同时，向上跳跃的时候，应该从当前点跳跃到链顶端的点的父亲结点，不能只跳到链顶去，因为是从一条链跳转到另一条链，而不是一条链内部跳跃。



# 树链剖分

```
while(top[u]!=top[v])//说明此时在一条链上了
{
    if(deep[top[u]]>deep[top[v]])//深度深的跳
        u=fa[top[u]];
    else
        v=fa[top[v]];
}
//最后两点在一条链上，深度浅的那个点就是最近公共祖先
return deep[u]<deep[v]?u:v;
```

注意是两点在一条链上的时候就不需要再跳了，而不是跳到同一个点之后在停止，当两点在同一条链上的时候，那么深度深的点一定可以向上跳到深度浅的点，那么深度浅的点就是最近公共祖先

# 树链剖分

根据前面我们的分析，如果要用树链剖分求lca需要记录几个参数：

**size[u]**: 以当前结点为根的子树结点之和

**son[u]**: 当前结点的重儿子的编号

**top[u]**: 当前结点所在链的顶端结点

**deep[u]**: 当前结点深度

**fa[u]**: 当前结点的父亲结点

跑一遍dfs可以确定size数组，deep数组，fa数组，递归返回的时候可以确定son数组，但是一遍dfs没办法确定top数组，因为返回的时候才确定那个是重儿子，所以对于后面的点他的链的顶点是暂时没有办法记录的。所以我们需要跑两遍dfs，第二遍确定top数组。

# 树链剖分

```
void dfs1(int u)//第一次求fa数组, size和重儿子
{
    size[u]=1;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa[u])
        {
            fa[v]=u;
            deep[v]=deep[u]+1;
            dfs1(v);
            size[u]+=size[v];
            if(!son[u]||size[v]>size[son[u]])
                son[u]=v;
        }
    }
}
```

# 树链剖分

```
void dfs2(int u,int tp)//第二次轻重链剖分
{
    top[u]=tp;//根结点肯定是一条重链顶点
    if(son[u])//先顺着重儿子找下去是一条重链, 顶点不变
        dfs2(son[u],tp);
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa[u]&&v!=son[u])//对于轻儿子, 链顶点就是它本身
            dfs2(v,v);
    }
}
```

# 树链剖分

前面就是如何用树链剖分求解lca问题的代码，主要思路就是每次跳跃都是从一条链跳跃到另一条上去(连续的重链看做是一条完整的链)，这样由于任意两个点之间最多隔了 $\log n$ 条链，所以最多 $\log n$ 次就一定可以跳到最近公共祖先上去。

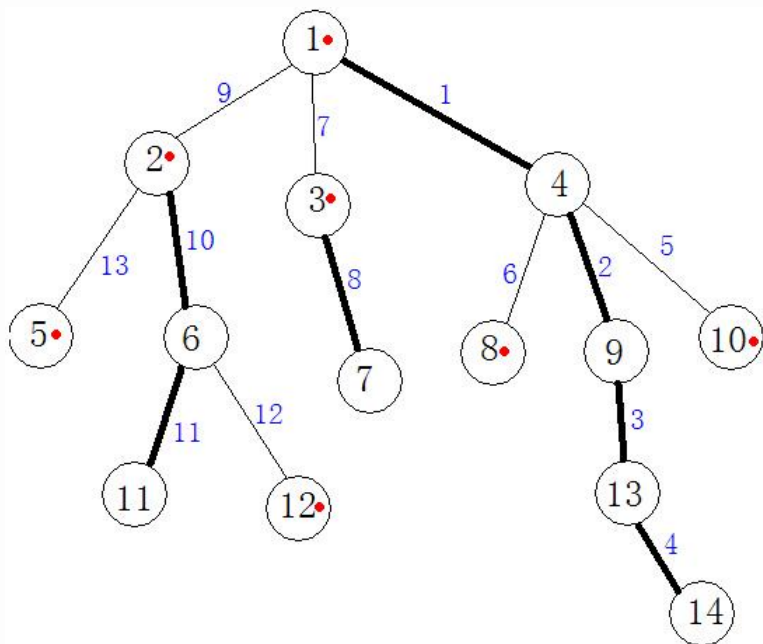
树链不仅仅是用来求解lca，它主要是用来求解带修改的树上路径，比如单点(边)修改，任意一条链修改，任意一棵子树修改。

当我们查询任意两点之间的路径的时候，实际上就是两点之间的所有轻重链之和，轻链就是一条轻边，但是重链可能是一条很长的链，或者两点之间的路径只是重链的一部分，我们应该如何记录呢？

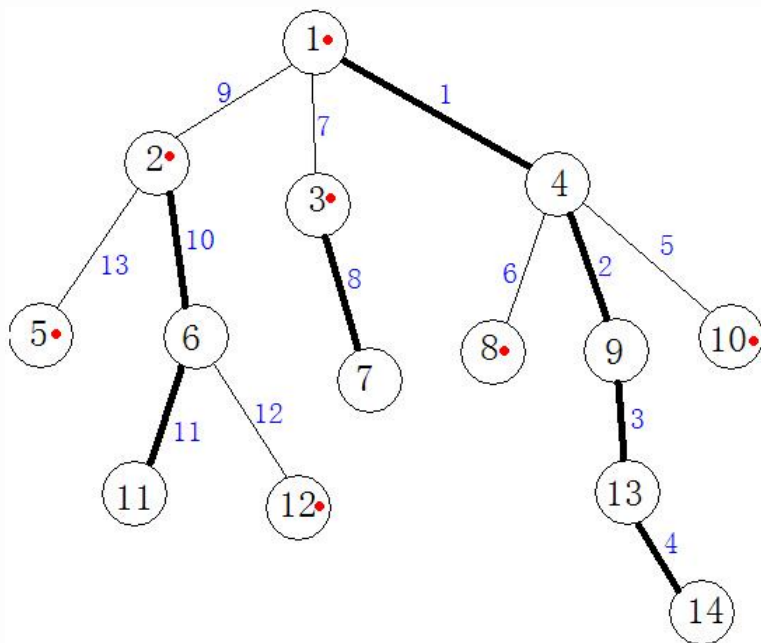
# 树链剖分

其实我们可以把树上问题转换成区间问题来求解，既然重链我们可以看做一个整体，但是有的时候又需要只计算部分，那么我们不妨把这些链从树上拿出来重新标记，也就是把一棵树重新拆分成链，把树上问题转换成区间问题。

那我们就对每一条边重新编号，重链上的边编号连续，轻链都是独立的，不影响。



# 树链剖分



这种编号方式有什么好处呢？

第一是重链是连续的区间，那么查询整条重链或者重链的连续部分也是一个连续区间，也就是区间求和

第二是一棵子树总体是一个连续的区间，那么如果我们查询子树之和，实际上也是区间求和。

# 树链剖分

那么如果涉及修改呢？这就很容易想到了，单点修改，就去修改那一个点就可以了，链修改，只需要修改经过的链就可以了，子树修改只需要修改连续的区间。实际上所有的修改都是修改一段或者多段连续的区间。所以树上修改操作我们可以用树链剖分把树拆分成链之和用区间修改区间查询来完成。那么就用简单的线段树就可以完成了。

就算是链修改，我们前面证明过，两点之间的链最多有 $\log n$ 条，那么最多修改 $\log n$ 个区间，每次修改区间时间复杂度是 $\log n$ ，那么最坏的总体时间复杂度是 $n \log n \log n$ ，还是远远低于 $n^2$ 。

这就是树链剖分+线段树来求解带树上修改的问题。



# 轻重链划分

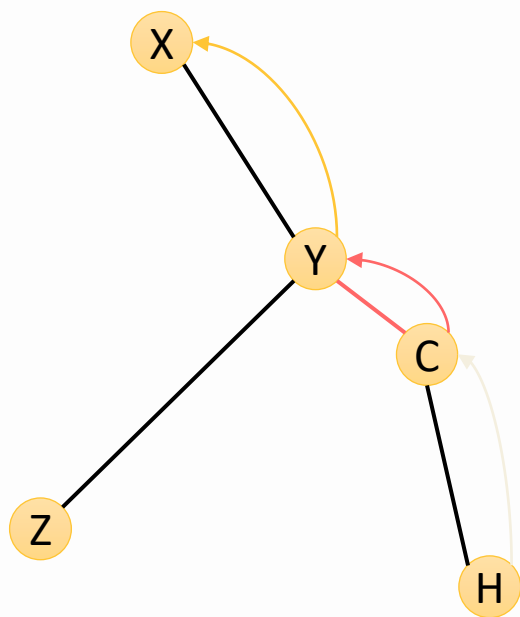
```
void dfs2(int u,int tp)
{
    top[u]=tp;
    num++;
    pos[u]=num;//原来的点x新赋予的编号为num
    tid[num]=u;//现在编号为num的点原来的编号
    if(son[u]==0)//如果是叶子结点
        return ;
    dfs2(son[u],tp);//tp表示当前重链的顶点。
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=son[u]&&v!=fa[u])
        {
            dfs2(v,v);//轻链的起点就是它本身
        }
    }
}
```

# 建树

接下来就很简单了啊，建一颗sum线段树，建法和以前一摸一样：但是要注意线段树中的区间标号代表的是线段的编号，所以对于每一个位置 $x$ 的值，实际上是 $tid[x]$ 的值。

# 分割

假设题目中要求我们修改 $u \rightarrow v$ 的路径，但是很大的可能性， $u \rightarrow v$ 没有在一条重链上，而是由多条重链和轻链组成的。所以我们需要先分割出来再去用线段树去查询。



比如我要修改 $Z \rightarrow H$ ，但是这两个点又不是一条重链上，我们可以参照lca的办法，都往上跳，准确的说，是深度大的往深度小的跳，如果是重儿子直接跳到重链的顶端，如果是轻儿子跳到父亲节点。最终一定会相遇。

# 分割

```
void trim(int x,int y,long long z)
{
    while(top[x]!=top[y])//如果x,y不在一条重链上
    {
        if(deep[top[x]]<deep[top[y]])
        {
            int tmp=x;x=y;y=tmp;
        }
        update(1,pos[top[x]],pos[x],z);//查询跳的这一段sum
        x=fa[top[x]];//说明上面是轻链了，找父亲节点
    }
    if(deep[x]>deep[y])//说明此时在一条重链上
    {
        int tmp=x;x=y;y=tmp;
    }
    update(1,pos[x],pos[y],z);
}
```

# 查询

如果要查询区间 $[l, r]$ ，其实和前面的修改是一样的，都是先找到当前区间 $[l, r]$ 在线段树中是由那几段区间构成的。只是把线段树的修改函数改成线段树的查询函数就可以了。

```
long long find(int x,int y)
{
    long long sum=0;
    while(top[x]!=top[y])
    {
        if(deep[top[x]]<deep[top[y]])
        {
            int tmp=x;
            x=y;
            y=tmp;
        }
        sum=(sum+query(1,pos[top[x]],pos[x]))%p;
        x=fa[top[x]];
    }
    if(deep[x]>deep[y])
    {
        int tmp=x;
        x=y;
        y=tmp;
    }
    sum=(sum+query(1,pos[x],pos[y]))%p;
    return sum;
}
```

# 子树修改

树链剖分是支持子树修改的，也就是修改以 $u$ 为根的整棵子树，比如子树的值都 $+x$ 。

我们在前面定义编号的时候，重链的编号是连续的，因为我们是dfs，所以实际上每个子树的编号总的来说也是连续的，所以实际上 $u$ 的子树在线段树上的区间就是 $[\text{pos}[u], \text{pos}[u] + \text{size}[u] - 1]$

# 树链剖分

前面我们讲解的树链剖分都是点修改，但是如果涉及的是树上边权修改，应该如何做呢？

首先我们要注意两点之间的路径点的数量比边的数量少一个，而且一个结点有多个儿子结点，但是只有一个父亲节点，所以我们可以用该结点代替该结点与其父亲结点相连的边。那么对于两点的最近公共祖先这个点代表的边就是其上方的边，并不在两点之间的路径上，所以加的时候最近公共祖先这个结点的值是不能加进去的。

计算方法有两种，第一种是和前面一样做加法，最后再把最近公共祖先减去。其实还有一种更加巧妙的办法，我们在最后一次计算的时候，是在一条链上计算， $news[y] - news[x]$ ，其中 $news[x]$ 就是最近公共祖先，那么我们只需要这个点不算就行，也就是 $[news[y]+1, news[x]]$ 。但是一定要注意轻链的情况，要提前结束



# 树链剖分

void trim(int x,int y)//注意修改的边只有 $k-1$ 条, 最近公共祖先那条边不修改

```
{
    while(top[x]!=top[y])
    {
        if(deep[top[x]]<deep[top[y]])
        {
            int tmp=x;x=y;y=tmp;
        }
        update(1,news[top[x]],news[x],1);
        x=fa[top[x]];
    }
    if(x==y)//特判, 防止 $l>r$ 
    return ;
    if(deep[x]<deep[y])
    {
        int now=x;x=y;y=now;
    }
    update(1,news[y]+1,news[x],1);//y这个点就是最近公共祖先
}
```



# 练习题

模板：

洛谷 3384 轻重链剖分(点权)

洛谷 3038 Grass Planting G(边权)

洛谷 3178 树上操作





# 练习题

洛谷 P1505 旅游  
洛谷 P2146 软件包管理器  
洛谷 P2486 染色  
洛谷 P2590 树的统计  
洛谷 P3250 网络  
洛谷 P3313 旅行  
洛谷 P3703 树点涂色

